

8. Programs for Virtual Functions

8.a Write a C++ Program To demonstrate method overriding using a virtual function

Aim:

To demonstrate method overriding using a virtual function.

Algorithm:

1. Start the program.
2. Create a base class with a virtual function.
3. Create a derived class and override the function.
4. Create a base class pointer pointing to derived object.
5. Call the function → executes derived class function.
6. End the program.

Program:

```
#include <iostream>
using namespace std;
class Base {
    public:
        virtual void display() {
            cout << "Base class function";
        }
};

class Derived : public Base
{
    public:
        void display()
        {
            cout << "Derived class function";
        }
};

int main() {
    Base *ptr;
    Derived d;
    ptr = &d;
    ptr->display();
    return 0;
}
```

Output

Derived class function

Result

Thus the C++ program has been executed successfully to demonstrate the method overriding using virtual function.

8.b Write a C++ Program to show that virtual destructor ensures proper object cleanup

Aim:

To show that virtual destructor ensures proper object cleanup.

Algorithm:

1. Start the program.
2. Create base class with virtual destructor.
3. Create derived class with destructor.
4. Create base pointer → assign derived object.
5. Delete pointer.
6. Observe call order of destructors.
7. End the program.

Program:

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual ~Base() {
        cout << "Base Destructor\n";
    }
};

class Derived : public Base {
public:
    ~Derived() {
        cout << "Derived Destructor\n";
    }
};

int main() {
    Base *ptr = new Derived;
    delete ptr;
    return 0;
}
```

Output:

Derived Destructor
Base Destructor

Result

Thus the C++ program has been executed successfully to show that virtual destructor ensures proper object cleanup